Summarize

Chapter 1. Introduction

Today's world of always-on applications and APIs has availability and reliability requirements that would have been required of only a handful of mission-critical services around the globe only a few decades ago. Likewise, the potential for rapid, viral growth of a service means that every application has to be built to scale nearly instantly in response to user demand. These constraints and requirements mean that almost every application that is built—whether it is a consumer mobile app or a backend payments application—needs to be a distributed system.

But building distributed systems is challenging. Often, they are one-off bespoke solutions. In this way, distributed system development bears a striking resemblance to the world of software development prior to the development of modern object-oriented programming languages. Fortunately, as with the development of object-oriented languages, there have been technological advances that have dramatically reduced the challenges of building distributed systems. In this case, it is the rising popularity of containers and container orchestrators. As with the concept of objects within object-oriented programming, these containerized building blocks are the basis for the development of reusable components and patterns that dramatically simplify and make accessible the practices of building reliable distributed systems. In the following introduction, we give a brief history of the developments that have led to where we are today.

A Brief History of Systems Development

In the beginning, there were machines built for specific purposes, such as calculating artillery tables or the tides, breaking codes, or other precise, complicated but rote mathematical applications. Eventually these purpose-built machines evolved into general-purpose programmable machines. And eventually they evolved from running one program at a time to running multiple programs on a single machine via time-sharing operating systems, but these machines were still disjoint from each other.

Gradually, machines came to be networked together, and client-server architectures were born so that a relatively low-powered machine on someone's desk could be used to harness the greater power of a mainframe in another room or building. While this sort of client-server programming was somewhat more complicated than writing a program for a single machine, it was still fairly straightforward to understand. The client(s) made requests; the server(s) serviced those requests.

In the early 2000s, the rise of the internet and large-scale data centers consisting of thousands of relatively low-cost commodity computers networked together gave rise to the widespread development of *distributed systems*. Unlike client-server architectures, distributed system applications are made up of multiple different applications running on different machines, or many replicas running across different machines, all communicating together to implement a system like web search or a retail sales platform.

Because of their distributed nature, when structured properly, distributed systems are inherently more reliable. And when architected correctly, they can lead to much more scalable organizational models for the teams of software engineers that built these systems. Unfortunately, these advantages come at a cost. These distributed systems can be significantly more complicated to design, build, and debug correctly. The engineering skills needed to build a reliable distributed system are significantly higher than those needed to build single-machine applications like mobile or web frontends. Regardless, the need for reliable distributed systems only continues to grow. Thus, there is a corresponding need for the tools, patterns, and practices for building them.

Fortunately, technology has also increased the ease with which you can build distributed systems. Containers, container images, and container orchestrators have all become popular in recent years because they are the foundation and building blocks for reliable distributed systems. Using containers and container orchestration as a foundation, we can establish a collection of patterns and reusable components. These patterns and components are a toolkit that we can use to build our systems more reliably and efficiently.

A Brief History of Patterns in Software Development

This is not the first time such a transformation has occurred in the software industry. For a better context on how patterns, practices, and reusable components have previously reshaped systems development, it is helpful to look at past moments when similar transformations have taken place.

Formalization of Algorithmic Programming

Though people had been programming for more than a decade before its publication in 1962, Donald Knuth's collection, *The Art of Computer Programming* (Addison-Wesley Professional), marks an important chapter in the development of computer science. In particular, the books contain algorithms not designed for any specific computer, but rather to educate the reader on the algorithms themselves. These algorithms then could be adapted to the specific architecture of the machine being used or the specific problem that the reader was solving. This formalization was important because it provided users with a shared toolkit for building their programs, but also because it showed that there was a general-purpose concept that programmers should learn and then subsequently apply in a variety of different contexts. The algorithms themselves, independent of any specific problem to solve, were worth understanding for their own sake.

Patterns for Object-Oriented Programming

Knuth's books represent an important landmark in the thinking about computer programming, and algorithms represent an important component in the development of computer programming. However, as the complexity of programs grew, and the number of people writing a single program grew from the single digits to the double digits and eventually to the thousands, it became clear that procedural programming languages and algorithms were insufficient for the tasks of modern-day programming. These changes in computer programming led to the development of object-oriented programming languages, which elevated data, reusability, and extensibility to peers of the algorithm in the development of computer programs.

In response to these changes to computer programming, there were changes to the patterns and practices for programming as well. Throughout the early to mid-1990s, there was an explosion of books on patterns for object-oriented programming. The most famous of these is the "gang of four" book, *Design Patterns: Elements of Reusable Object-Oriented Programming* by Erich Gamma et al. (Addison-Wesley Professional). *Design Patterns* gave a common language and framework to the task of programming. It described a series of interface-based patterns that could be reused in a variety of contexts. Because of advances in object-oriented programming and specifically interfaces, these patterns could also be implemented as generic reusable libraries. These libraries could be written once by a community of developers and reused repeatedly, saving time and improving reliability.

The Rise of Open Source Software

Though the concept of developers sharing source code has been around nearly since the beginning of computing, and formal free software organizations have been in existence since the mid-1980s, the very late 1990s and the 2000s saw a dramatic increase in the development and distribution of open source software. Though open source is only tangentially related to the development of patterns for distributed systems, it is important in the sense that it was through the open source communities that it became increasingly clear that software development in general and distributed systems development in particular are community endeavors. It is important to note that all of the container technology that forms the foundation of the patterns described in this book has been developed and released as open source software. The value of patterns for both describing and improving the practice of distributed development is especially clear when you look at it from this community perspective.

NOTE

What is a pattern for a distributed system? There are plenty of instructions out there that will tell you how to install specific distributed systems (such as a NoSQL database). Likewise, there are recipes for a specific collection of systems (like a MEAN stack). But when I speak of patterns, I'm referring to general blueprints for organizing distributed systems, without mandating any specific technology or application choices. The purpose of a pattern is to provide general advice or structure to guide your design. The hope is that such patterns will guide your thinking and also be generally applicable to a wide variety of applications and environments.

The Value of Patterns, Practices, and Components

Before spending any of your valuable time reading about a series of patterns that I claim will improve your development practices, teach you new skills, and—let's face it—change your life, it's reasonable to ask: "Why?" What is it about the design patterns and practices that can change the way that we design and build software? In this section, I'll lay out the reasons I think this is an important topic, and hopefully convince you to stick with me for the rest of the book.

Standing on the Shoulders of Giants

As a starting point, the value that patterns for distributed systems offer is the opportunity to figuratively stand on the shoulders of giants. It's rarely the case that the problems we solve or the systems we build are truly unique. Ultimately, the combination of pieces that we put together and the overall business model that the software enables may be something that the world has never seen before. But the way the system is built and the problems it encounters as it aspires to be reliable, agile, and scalable are not new.

This, then, is the first value of patterns: they allow us to learn from the mistakes of others. Perhaps you have never built a distributed system before, or perhaps you have never built this type of distributed system. Rather than hoping that a colleague has some experience in this area or learning by making the same mistakes that others have already made, you can turn to patterns as your guide. Learning about patterns for distributed system development is the same as learning about any other best practice in computer programming. It accelerates your ability to build software without requiring that you have direct experience with the systems, mistakes, and firsthand learning that led to the codification of the pattern in the first place.

A Shared Language for Discussing Our Practice

Learning about and accelerating our understanding of distributed systems is only the first value of having a shared set of patterns. Patterns have value even for experienced distributed system developers who already understand them well. Patterns provide a shared vocabulary that enables us to understand each other quickly. This understanding forms the basis for knowledge sharing and further learning.

To better understand this, imagine that we both are using the same object to build our house. I call that object a "Foo" while you call that object a "Bar." How long will we spend arguing about the value of a Foo versus that of a Bar, or trying to explain the differing properties of Foo and Bar until we figure out that we're speaking about the same object? Only once we determine that Foo and Bar are the same can we truly start learning from each other's experience.

Without a common vocabulary, we waste time in arguments of "violent agreement" or in explaining concepts that others understand but know by another name. Consequently, another significant value of patterns is to provide a common set of names and definitions so that we don't waste time worrying about naming, and instead get right down to discussing the details and implementation of the core concepts.

I have seen this happen in my short time working on containers. Along the way, the notion of a *sidecar* container (described in Chapter 3 of this book) took hold within the container community. Because of this, we no longer have to spend time defining what it means to be a sidecar and can instead jump immediately to how the concept can be used to solve a particular problem. "If we just use a sidecar" ... "Yeah, and I know just the container we can use for that." This example leads to the third value of patterns: the construction of reusable components.

Shared Components for Easy Reuse

Beyond enabling people to learn from others and providing a shared vocabulary for discussing the art of building systems, patterns provide another important tool for computer programming: the ability to identify common components that can be implemented once.

If we had to create all of the code that our programs use ourselves, we would never get done. Indeed, we would barely get started. Today, every system ever written stands on the shoulders of thousands if not hundreds of thousands of years of human effort. Code for operating systems, printer drivers, distributed databases, container runtimes, and container orchestrators—indeed, the entirety of applications that we build today are built with reusable shared libraries and components.

Patterns are the basis for the definition and development of such reusable components. The formalization of algorithms led to reusable implementations of sorting and other canonical algorithms. The identification of interface-based patterns gave rise to a collection of generic object-oriented libraries implementing those patterns.

Identifying core patterns for distributed systems enables us to build shared common components. Implementing these patterns as container images with HTTP-based interfaces means they can be reused across many different programming languages. And, of course, building reusable components improves the quality of each component because the shared codebase gets sufficient usage to identify bugs and weaknesses, and sufficient attention to ensure that they get fixed.

More recently, a series of software supply chain attacks has made dependencies and dependency management a critical part of securing our applications. In the context of a secure software supply chain, these shared components take on even more importance. Every library or application that we use brings in more dependencies—and consequently, more risk. Relying on a single shared implementation of a core idea reduces the total amount of software that the world needs to depend on, and by focusing attention on a

few dependencies, significantly improves the chances that they are properly maintained and protected from software supply chain attacks.

Summary

Distributed systems are required to implement the level of reliability, agility, and scale expected of modern computer programs. Distributed system design continues to be more of a black art practiced by wizards than a science applied by laypeople. The identification of common patterns and practices has regularized and improved the practice of algorithmic development and object-oriented programming. It is this book's goal to do the same for distributed systems. Enjoy!

Previous chapter

< I. Foundational Concepts

Next chapter

2. Important Distributed System Concepts >